

Paradigmi di Programmazione - A.A. 2022-23

Esame Scritto del 14/07/2023

CRITERI DI VALUTAZIONE:

La prova è superata se si ottengono almeno 12 punti negli esercizi 1,2,3 e almeno 18 punti complessivamente.

Esercizio 1 [Punti 4]

Applicare la β -riduzione alla seguente λ -espressione fino a raggiungere una espressione non ulteriormente riducibile o ad accorgersi che la derivazione è infinita.

$$(\lambda x. \lambda y. xy)(\lambda x. (((\lambda z. z)x)y))$$

Nella soluzione, mostrare tutti i passi di riduzione calcolati, sottolineando ad ogni passo la porzione di espressione a cui si applica la β -riduzione (redex) ed evidenziando le eventuali α -conversioni.

SOLUZIONE:

Una possibile soluzione:

$$\begin{aligned} & (\lambda x. \lambda y. xy)(\lambda x. (((\lambda z. z)x)y)) \\ \rightarrow & \quad (\lambda x. \lambda y. xy)(\lambda x. xy) \\ \equiv_{\alpha} & \quad (\lambda x. \lambda k. xk)(\lambda x. xy) \\ \rightarrow & \quad (\lambda k. (\lambda x. xy)k) \\ \rightarrow & \quad \lambda k. ky \end{aligned}$$

Esercizio 2 [Punti 4]

Indicare il tipo della seguente funzione OCaml, mostrando i passi fatti per inferirlo:

```
let f x y z =  
  let g = fun k -> x+k in  
  match y with  
  | (a,0) -> if a=z then g 1 else g 2  
  | (a,n) -> g n
```

SOLUZIONE:

Struttura del tipo:

```
X -> Y -> Z -> RIS
```

Uso per convenzione X, Y, Z come variabili di tipo per i parametri x, y, z , RIS come variabile di tipo del risultato, e $A, B, C, \dots$ come variabili di tipo "fresche" per la definizione dei vincoli.

Vincoli:

```
X = int           (da definizione di g)
G = int -> int     (da definizione di g)
Y = A * int        (da primo caso del pattern matching)
A = Z              (da guardia dell'if)
RIS = int          (da casi del pattern matching)
```

Ne consegue:

```
X = int
Y = A * int
Z = A
RIS = int
```

Tipo inferito:

```
int -> (A * int) -> A -> int
```

in sintassi OCaml:

```
int -> ('a * int) -> 'a -> int
```

Esercizio 3 [Punti 7]

Definire in OCaml, usando i costrutti di programmazione funzionale, le seguenti funzioni:

```
bimedia : int*int list -> int*int
```

```
alternati : int*int list -> int
```

tali che:

- **bimedia** prende una lista di coppie di interi `lis` che si assume non vuota e restituisce la coppia $(m1, m2)$ dove $m1$ è la media dei primi elementi delle coppie in `lis` ed $m2$ è la media dei secondi elementi delle stesse coppie. Ad esempio:

```
bimedia [(1,20);(5,10);(6,30)]    (* restituisce (4,20) *)
```

- **alternati** prende una lista di coppie di interi `lis` e restituisce la somma dei primi elementi delle coppie in posizioni dispari nella lista con i secondi elementi delle coppie in posizioni pari. In altre parole, se `lis = [(a1,b1);(a2,b2);(a3,b3);(a4,b4);...]` calcola $a1+b2+a3+b4+\dots$. Ad esempio:

```
alternati [(1,10);(2,20);(3,30);(4,40)]    (* restituisce 64 *)
```

SOLUZIONE:

Una possibile soluzione:

```
let bimedia lis =
  let f (sa,sb,c) (a,b) = (sa+a,sb+b,c+1) in
  let (tot_a,tot_b,cont) = List.fold_left f (0,0,0) lis in
  (tot_a/cont,tot_b/cont);;

let rec alternati lis =
  match lis with
  | [] -> 0
  | (a,b)::lis' -> a + alternati2 lis'
and alternati2 lis =
  match lis with
  | [] -> 0
  | (a,b)::lis' -> b + alternati lis'
```

Esercizio 4 [Punti 15]

Si estenda il linguaggio MiniCaml visto a lezione con il costrutto **PartFun** per la definizione di *funzioni parziali*, ossia funzioni che dato un parametro formale di un certo tipo (qualunque), potrebbero non essere utilizzabili su alcuni valori di quel tipo. Il costrutto prevede un predicato che consente di verificare se la funzione sia definita per un certo valore del parametro. Al momento dell'applicazione, se il predicato è vero viene valutato il corpo, altrimenti viene sollevata un'eccezione. La sintassi astratta del costrutto è quindi:

$$\text{PartFun } x \rightarrow \text{pred} : \text{body}$$

dove **x** è il parametro formale, **pred** è il predicato e **body** è il corpo della funzione.

Ad esempio, usando una sintassi simile a OCaml, la seguente funzione parziale è definita solo per valori diversi da zero, e calcola $100/x$:

```
part_fun x -> ( x<>0 ) : ( 100/x )
```

e la sua applicazione:

```
( part_fun x -> ( x<>0 ) : ( 100/x ) ) 5          (* restituisce 20 *)
( part_fun x -> ( x<>0 ) : ( 100/x ) ) 0          (* solleva eccezione *)
```

Altro esempio, la seguente funzione parziale è definita solo per valori pari e calcola $x/2$ (nota: il corpo della funzione potrebbe essere eseguito anche su valori dispari, ma il predicato consente di usarla solo su pari):

```
part_fun x -> ( x%2==0 ) : ( x/2 )
```

e la sua applicazione:

```
( part_fun x -> ( x%2==0 ) : ( x/2 ) ) 10          (* restituisce 5 *)
( part_fun x -> ( x%2==0 ) : ( x/2 ) ) 5          (* solleva eccezione *)
```

Per la gestione delle funzioni parziali, il linguaggio deve essere esteso anche con i seguenti due costrutti:

- **Test**: data una funzione parziale e un valore, restituisce **true** se la funzione è definita per quel valore e **false** altrimenti, senza applicarla.

- **Force:** data una funzione parziale e un valore, applica la funzione (valuta il corpo) senza valutare prima il predicato

Esempi (in sintassi simili-OCaml) di **Test** e **Force** con le due funzioni definite sopra:

```
test ( part_fun x -> ( x<>0 ) : ( 100/x ) ) 5      (* restituisce true *)
test ( part_fun x -> ( x<>0 ) : ( 100/x ) ) 0      (* restituisce false *)
force ( part_fun x -> ( x<>0 ) : ( 100/x ) ) 5      (* restituisce 20 *)
force ( part_fun x -> ( x<>0 ) : ( 100/x ) ) 0      (* errore dato dal calcolo di 100/0 *)

test ( part_fun x -> ( x%2==0 ) : ( x/2 ) ) 10      (* restituisce true *)
test ( part_fun x -> ( x%2==0 ) : ( x/2 ) ) 5      (* restituisce false *)
force ( part_fun x -> ( x%2==0 ) : ( x/2 ) ) 10      (* restituisce 5 *)
force ( part_fun x -> ( x%2==0 ) : ( x/2 ) ) 5      (* restituisce 2 *)
```

Si mostri come deve essere modificato l'interprete OCaml del linguaggio.

SOLUZIONE:

Una possibile soluzione:

```
type exp = ...
  | PartFun of ide * exp * exp
  | Test of exp * exp
  | Force of exp * exp

type evT = ...
  | PartClosure of ide * exp * exp * evT env

let rec eval e s = match e with
  ...
  | PartFun (arg, pred, body) -> PartClosure (arg,pred,body,s)
  | Apply (eF,e) ->
    let fclosure = eval eF s in
    (match fclosure with
      ....
      | PartClosure (arg,pred,body,fDecEnv) ->
        let aval = eval e s in
        let aenv = bind fDecEnv arg aval in
        (match (eval pred aenv) with
          | Bool(true) -> eval body aenv
          | Bool(false) -> failwith "Non definita"
        )
      ....
    )
  | Test (eF,e) ->
    let fclosure = eval eF s in
    (match fclosure with
      | PartClosure (arg,pred,body,fDecEnv) ->
        let aval = eval e s in
        let aenv = bind fDecEnv arg aval in
        eval pred aenv
      | _ -> failwith "Error"
    )
  | Force (eF,e) ->
    let fclosure = eval eF s in
    (match fclosure with
      | PartClosure (arg,pred,body,fDecEnv) ->
        let aval = eval e s in
        let aenv = bind fDecEnv arg aval in
        eval body aenv
      | _ -> failwith "Error"
    )
  )
```